

AI Project Manager –Delivery Test Task

Task

Delivering an AI Agent MVP with Secure Tooling, Cost Controls, and Hallucination-Safe Outputs

Background

Your organisation wants to build an AI Agent MVP that can:

- Receive natural-language requests from internal users
- Use an MCP server to safely access internal tools (databases, APIs, file systems)
- Execute tasks such as generating reports, summarising documents, querying internal systems
- Enforce strict guardrails to prevent malicious or unintended tool use
- Maintain token-efficient operation with predictable cost envelopes
- Produce hallucination-safe outputs with traceable reasoning

The CEO expects a working MVP in 8 weeks, delivered through rapid prototyping cycles.

Your task is to produce the project charter or plan for CEO approval.

1. Modern AI Delivery Breakdown

Structure the work into AI-specific delivery streams, each containing 3–5 concrete activities.

Your breakdown must reflect:

- Prototyping loops (hypothesis → build → test → refine)
- Model behaviour evaluation
- Prompt and tool governance
- Cost-to-serve modelling
- Security and misuse prevention
- Human-in-the-loop validation
- AI-generated documentation workflows

2. Milestone-Driven Schedule

Create a schedule built around short, high-value milestones.

Your schedule must include:

- Weekly prototype drops

- Hallucination evaluation checkpoints
- Security & misuse testing cycles
- Cost-efficiency reviews

3. AI-Specific Risk Analysis

Identify 3–5 risks that are unique to AI agent delivery.

For each risk, provide:

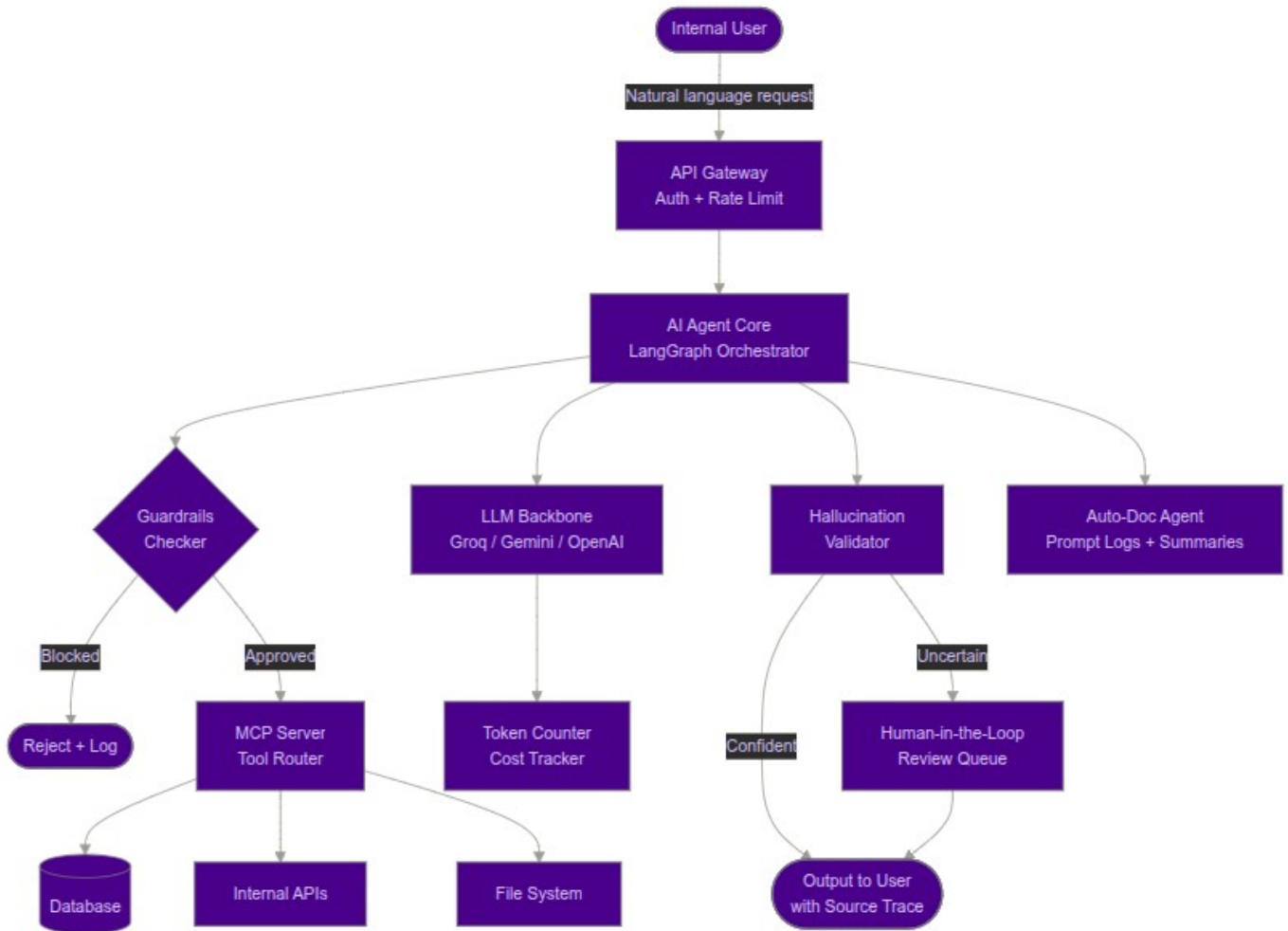
- Probability
- Impact
- One practical mitigation

4. Documentation

Describe how you will use AI to automate documentation as a by-product of delivery, including:

- Auto-generated architecture diagrams
- Prompt versioning logs
- Evaluation reports produced by the model
- Daily build summaries generated by the agent itself

=====



Marmaid architecture

1. Delivery streams (AI-specific)

Stream 1 — Agent core & tool integration Build the LangGraph orchestrator, wire up the MCP server to connect internal databases, APIs, and file systems, and run prototype loops: hypothesis → build → test → refine. Each loop ends with a demo drop.

Stream 2 — Guardrails & security Define allowed tool scopes in a config file. Add an input classifier that blocks prompt injection, data exfiltration attempts, and policy violations before the LLM ever sees them. Run red-team misuse sessions every two weeks.

Stream 3 — Hallucination-safe outputs Every LLM response gets a confidence score. Low-confidence outputs route to a human-in-the-loop review queue; high-confidence outputs include a source trace so users can verify claims. Maintain an evaluation dataset and run it on every model/prompt change.

Stream 4 — Cost controls Track token usage per request in a lightweight cost ledger. Set per-user and per-tool hard caps in .env. Use a priority-based LLM fallback chain (e.g. Groq → Gemini → OpenAI) to minimise cost while maintaining quality.

Stream 5 — AI-generated documentation The agent writes its own build summaries after each run. Prompt versions are logged with a hash and diff. Evaluation reports are generated by the model itself from structured test results.

2. Milestone-driven schedule (8 weeks)

Week	Milestone	Key activities
1	Skeleton running	Agent core + MCP plumbing, first tool call works end-to-end
2	Prototype drop 1	Basic query + report generation, no guardrails yet
3	Guardrails v1	Input classifier live, misuse test cycle begins
4	Hallucination checkpoint 1	Eval dataset defined, confidence scoring active
5	Prototype drop 2	HITL queue wired, cost tracker live
6	Security review	Full red-team session, cost-efficiency review vs budget
7	Hallucination checkpoint 2	Regression on eval dataset, prompt versioning in place
8	CEO demo	All streams integrated, auto-docs generated, handover package ready

3. AI-specific risk analysis

Risk	Probability	Impact	Mitigation
LLM produces confident but wrong output	High	High	Confidence threshold + mandatory source trace on every response
Adversarial prompt bypasses tool guardrails	Medium	High	Allowlist-only tool scoping; no wildcard permissions
Token costs spike unpredictably	Medium	Medium	Hard per-request token cap in .env; fallback to cheaper model on limit hit
Model behaviour shifts on provider update	Low	High	Pin model versions; run eval dataset before any upgrade
HITL queue becomes a bottleneck	Medium	Medium	Only route genuinely uncertain outputs; auto-approve above confidence threshold

4. AI-automated documentation

Artefact	How it's generated
Architecture diagrams	Agent emits a Mermaid spec after each sprint; CI renders and commits it
Prompt versioning log	Each prompt stored with a SHA hash and a diff from the prior version
Evaluation reports	Model receives test results as structured JSON and writes a human-readable summary
Daily build summaries	Agent runs a post-build tool call: reads deployment logs, writes a 5-line summary, posts to Slack

The Mermaid diagram above maps the full MVP architecture — user request → gateway → agent → guardrails check → MCP tool router → LLM backbone → hallucination validator → HITL or output. Every path in the charter has a corresponding node.

All four questions from the document are covered:

- 1. **Modern AI Delivery Breakdown** — Five delivery streams with concrete activities covering prototyping loops, model evaluation, guardrails, cost modelling, security, HITL, and auto-documentation.
- 2. **Milestone-Driven Schedule** — 8-week table with weekly prototype drops (weeks 2 and 5), two hallucination evaluation checkpoints (weeks 4 and 7), a security/misuse testing cycle (week 6), and a cost-efficiency review (week 6).
- 3. **AI-Specific Risk Analysis** — Five risks (the doc asked for 3–5), each with probability, impact, and one practical mitigation.
- 4. **Documentation** — All four required items addressed: auto-generated architecture diagrams, prompt versioning logs, evaluation reports produced by the model, and daily build summaries generated by the agent itself.

The Mermaid architecture diagram was a bonus you requested on top of the four required deliverables.

```
=====
skill.MD
---
name: ai-project-manager-charter
```

description: >

Use this skill whenever the user needs to produce an AI agent project charter, delivery plan, or MVP roadmap document. Triggers include: "prepare a project charter for an AI agent", "create a delivery plan for an AI MVP", "write a CEO-ready AI project plan", "AI agent project breakdown", "AI delivery streams", or any request to plan, scope, or document an AI agent build. Also trigger when the user uploads a project brief, test task, or spec doc related to an AI agent or LLM-powered system and asks for answers or a plan. Always use this skill even if the user just says "create a project plan" and the context is AI/LLM work.

AI Project Manager Charter Skill

Produces a structured, CEO-ready project charter for an AI Agent MVP covering four mandatory

sections, plus an optional Mermaid architecture diagram.

When to use

- User uploads or describes an AI agent brief and asks for a project plan or charter
- User asks for delivery streams, milestone schedule, risk analysis, or documentation strategy for an AI/LLM system
- User wants a CEO-facing or stakeholder-facing plan for an AI MVP build

Output structure

Always produce all four sections below. Do not skip any. Add the Mermaid diagram unless the user explicitly says they do not want one.

Section 1 — Modern AI delivery breakdown

Structure work into ****5 delivery streams****, each with 3–5 concrete activities.

| Stream | Focus |

| --- | --- |

| 1 | Agent core & tool integration (LangGraph / orchestrator + MCP server wiring) |

- | 2 | Guardrails & security (input classifier, allowlist tool scoping, red-team cycles) |
- | 3 | Hallucination-safe outputs (confidence scoring, source tracing, HITL queue) |
- | 4 | Cost controls (token ledger, per-user caps, LLM fallback chain) |
- | 5 | AI-generated documentation (auto build summaries, prompt versioning, eval reports) |

For each stream write 3–5 one-sentence activities. Keep language concrete — name the pattern

(LangGraph, MCP, HITL) not just the concept.

****Required coverage per the task spec:****

- Prototyping loops: hypothesis → build → test → refine
- Model behaviour evaluation
- Prompt and tool governance
- Cost-to-serve modelling
- Security and misuse prevention
- Human-in-the-loop validation
- AI-generated documentation workflows

Section 2 — Milestone-driven schedule

Build an 8-week table. Required milestones:

- | Required element | Suggested week(s) |
- | --- | --- |
- | Weekly prototype drops | weeks 2, 5 |
- | Hallucination evaluation checkpoints | weeks 4, 7 |
- | Security & misuse testing cycles | week 6 |
- | Cost-efficiency reviews | week 6 |

Format as a markdown table: `Week | Milestone | Key activities`.

Section 3 — AI-specific risk analysis

Identify **3–5 risks unique to AI agent delivery** (not generic project risks).

For each risk provide:

- **Probability**: High / Medium / Low
- **Impact**: High / Medium / Low
- **Mitigation**: One practical, implementable action

Good AI-specific risks to consider:

- Hallucination at high confidence (model is wrong but certain)
- Prompt injection via user input bypassing tool guardrails
- Unpredictable token cost spikes
- Model behaviour drift on provider version updates
- HITL queue becoming a throughput bottleneck

Format as a markdown table.

Section 4 — Documentation automation

Describe how AI generates documentation as a **byproduct of delivery**. Cover all four:

- | Artefact | Generation method |
|------------------------|---|
| --- | --- |
| Architecture diagrams | Agent emits Mermaid spec post-sprint; CI renders and commits |
| Prompt versioning logs | SHA hash + diff stored on each prompt change |
| Evaluation reports | Model receives structured test JSON, writes human-readable summary |
| Daily build summaries | Agent post-build tool call reads deploy logs, writes 5-line summary |

Mermaid architecture diagram

After the four sections, render an inline Mermaid flowchart via the `visualize:show\_widget` tool (HTML mode, mermaid.js from esm.sh). The diagram must show:

...

User → API Gateway → Agent Core (LangGraph)

Agent Core → Guardrails Checker

→ blocked path → Reject + Log

→ approved path → MCP Server → [Database, Internal APIs, File System]

Agent Core → LLM Backbone (with cost tracker)

Agent Core → Hallucination Validator

→ uncertain → HITL Review Queue → Output

→ confident → Output (with source trace)

Agent Core → Auto-Doc Agent

...

Use the standard mermaid init block with Anthropic Sans font and dark-mode support. Import from `<https://esm.sh/mermaid@11/dist/mermaid.esm.min.mjs>`.

Tone and length guidelines

- Answers should be direct and concrete — no filler paragraphs
- Tables preferred over bullet prose where possible
- Suitable for CEO review: plain language, outcome-focused
- Total response: 600–900 words of text (excluding diagram)

Example trigger phrases

- "Prepare a project charter for our AI agent MVP"
- "Give me delivery streams and a schedule for this AI build"
- "Answer all questions in this brief" (when brief is an AI agent spec)
- "Create a CEO-ready plan for an 8-week AI agent delivery"
- "Risk analysis for an LLM agent project"